

02-614 Week 4

String Algorithms

Aidan Jan

September 22, 2026

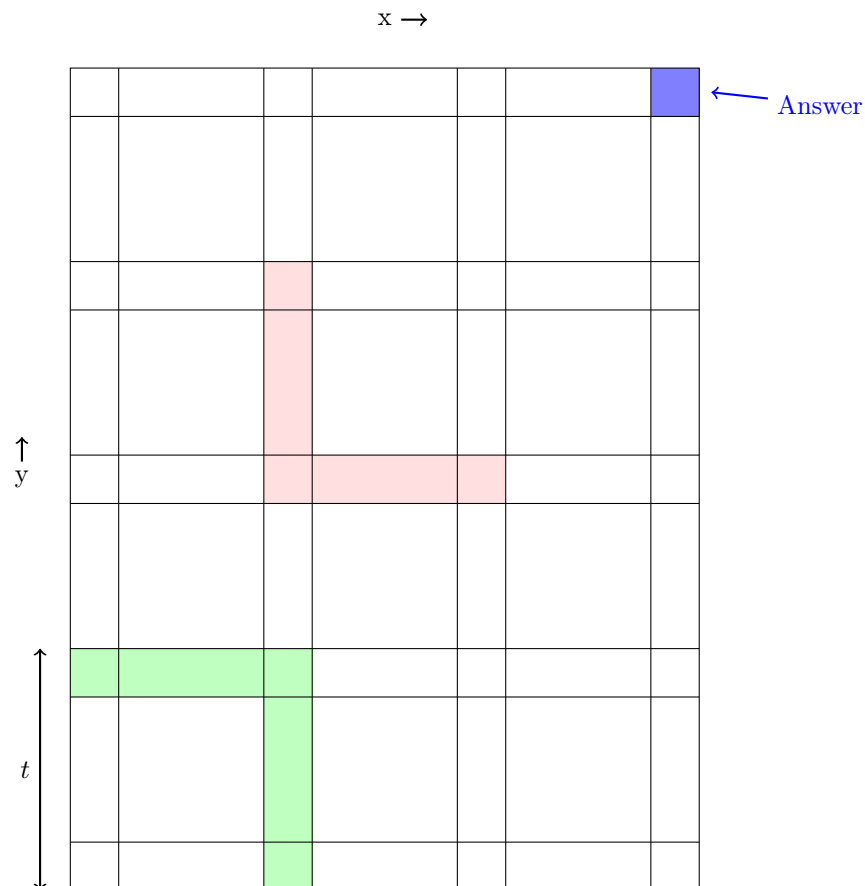
Four Russian's Speedup

There is a faster way to do edit distance than Hirschberg's algorithm that also keeps the same space complexity.

- **Goal:** Edit distance faster than $O(m, n)$ with no increase in space (asymptotically).

Idea 1:

Imagine we can tile our dynamic programming table with tiles of size t . Instead of calculating every cell individually (and take up mn time to do so), we can compute in larger chunks.



- Each tile overlaps the other tiles by 1 row and 1 column on each side.

- We start from the bottom left corner (which we call cell $(0,0)$). Consequently, our answer is the top right.

First, we have to make a few assumptions:

- $m = n$, or in other words, $|x| = |y|$.
- Our tiles exactly tile the DP matrix.

Idea 2:

With this tile setup, only the edges of the tiles matter.

- If we know the edges, then we don't need the centers of the tiles.

We want:

- A function $f(\text{red shaded edges}, x', y') \rightarrow \text{green shaded edges}$. In other words, take in the bottom and left edges of each tile and output the values of the top and right edges.

It turns out that we can do this using the Hirschberg or the Naïve dynamic programming algorithm. Therefore, a tile would take $O(t^2)$ time. Can we compute that faster?

Computing f Faster

Lemma: In the edit distance (number of operations) DP matrix D , adjacent cells in a row, column, or diagonal differ by at most 1.

		0	-1	-1	-1	← Output offset Vector
1		$c+2$	$c+1$	c	$c-1$	
1		$c+1$	$c+1$	c	$c-1$	
0		c	$c+2$	$c+3$	$c+4$	← Violation of the Lemma
0		c	$c+1$	c	c	
		0	1	-1	0	← Input offset Vector

Idea 3:

Represent the **red shaded edges** and **green shaded edges** as offset vectors.

- In the example above, we define the bottom offset vector at $(0, 1, -1, 0)$, since those are the differences between each of the values in the cells along the bottom rows
- Since every tile represents matching a substring between x and y and the bottom row and left columns represent the previous alignment, the best alignment (the output) always remains the same if the input offset vectors and the characters in the string are the same.
- This means we can precompute every f value and do lookups to fill in the edges of tiles on the DP table.

Idea 4:

Precompute every possible f value.

- (1) There are $3^{2(t-1)} \cdot |\Sigma|^{2t}$ possible inputs to f .
 - $3^{2(t-1)}$ comes from the fact that offset vectors must contain either $\{-1, 0, 1\}$ in each position, and there are a total of $2(t-1)$ positions along the left and bottom sides of a tile.
 - The $|\Sigma|^{2t}$ comes from the number of possible substrings that could be in x and y , for $2t$ characters total, and each character has $|\Sigma|$ choices.
- (2) Computing all possible values takes $O((3|\Sigma|)^{2t} \cdot t^2)$. (The t^2 is from the DP algorithm to fill in the cells of a tile once.)

However, the trick here is that t is an arbitrary tile size value, which we are able to set. We pick $t = \frac{1}{2} \left(\log_{3|\Sigma|} n \right)$.

- Now, if we plug in for t , our time complexity becomes $O(n(\log n)^2)$.

This pre-computation now takes less time than the rest of the problem because we only do it once.

Storing f_s

We make a tree that encodes the input and stores the output at the leaves.

- Start at the root
- The first $2(t-1)$ layers are all the possible offset vector combinations
- The next $2t$ layers are possible substrings.
- The output offset vectors (i.e., the top and right sides of each tile) are the data stored at the leaf nodes.

With this, searching up an f value is linear time, since our tree is $O(t)$ layers deep.

- In this case, the tree acts similarly to a hash map except without the $O(1)$ lookup. It maps an input key (input offset vectors and substrings) to an output value (output offset vectors).
- We don't use a hash map because the number of keys we have is exponential - a hash map would run into bucketing problems and become linear time (relative to the number of total possible inputs, i.e., exponential) lookup.
- The search time for a tree of $O(t)$ height would be $O(t)$, which is faster than all the other parts of the algorithm anyways.

Storing the Tree

When we store using the tree, traversing the tree takes $O(t) = O(\log n)$ time.

- However, instead if we create a table (a direct map from the input to the output), we can do constant time lookup.
- It turns out that $O(\log(n))$ bitstrings can be manipulated in $O(1)$ time.
- Every possible bitstring $O(2^{\log(n)}) = O(n)$ is stored in the table, with an n -sized table.

Total Running Time

- We have n^2/t^2 tiles.
- It takes $O(t)$ to solve each tile by using lookups in the tree.
- The preprocessing takes $O(n \log^2 n)$ time.

In total, we have $O(\frac{n^2}{t^2} \cdot t + n \log^2 n)$, which simplifies to $O(\frac{n^2}{t} + n \log^2 n)$. Since the size of t we determined is equal to $\log n$, we can substitute that in, and drop the second term. (The $\frac{n^2}{\log(n)}$ term dominates.)

$$\boxed{O\left(\frac{n^2}{\log n}\right)}$$

Traceback Path

- We traceback similarly to the naïve DP algorithm
- When calculating each of the tiles, we use the naïve algorithm, which means we also have the traceback arrows.
- Therefore, we can map each output position of a tile to an input position. This adds a constant amount of storage and can be stored in the same table.
- In the worst case, the traceback passes through $\frac{2n}{t} = O\left(\frac{2n}{\log n}\right)$ tiles, which involves going straight left then straight down.
- To find the exact traceback path, we then run the naïve DP algorithm only on the tiles that we pass through. The result is $O(\frac{2n}{t} \cdot t^2) = O(n \log n)$ time. (This term is dropped because the rest of the algorithm is slower.)

Multiple Sequence Alignment

Suppose we multiple sequences we want to align with each other. We can place these sequences into a matrix where each row is one string. This matrix is referred to as the **alignment matrix**.

- The goal of this alignment matrix is to add gaps to each string (so they all become the same length), and maximum **score**
- `colcost()` is a function that takes in a *column* of the alignment matrix and returns a number, the **score**, where in general, better aligned columns score higher than misaligned columns. Depending on the purposes of doing the alignment, this scoring function may differ.

OPT Matrix Calculation

$$\text{OPT}[i_1, i_2, \dots, i_p] = \min \begin{cases} \text{colcost}(_, c, c, \dots, c) + \text{OPT}[i_1, i_2 - 1, i_3 - 1, \dots, i_p - 1] \\ \text{colcost}(c, _, c, \dots, c) + \text{OPT}[i_1 - 1, i_2, i_3 - 1, \dots, i_p - 1] \\ \vdots \\ \text{colcost}(_, \dots, _, c) + \text{OPT}[i_1, i_2, \dots, i_{p-1}, i_p - 1] \end{cases}$$

- Essentially, the optimal for a given cell is equal to the minimum on the p -dimensional DP table of (score of column from inserting gaps at certain positions) plus the optimal based on the neighboring positions.
- If we inserted a gap at the given row in the alignment matrix, we use the **OPT** cell at the current row. If we don't insert a gap (represented by c in the `colcost` function above), we use the previous row.

- In total, the OPT recursive function would contain $O(2^p)$ rows, since we have that many possible ways to add gaps on a given column.
- It is similar to running single sequence alignment on every string, except we now need to consider every string at the same time.
- Note that this means this problem is NP-Hard in most formulations, because computing the DP matrix would take $\Omega(n^p 2^p)$ time. This number is hard to reduce. However, we may use approximation algorithms for a polynomial time solution, even though they cannot offer us the optimal score.

Heuristic: Progressive Alignment

Instead of aligning all the strings at the same time, we can instead align each string separately.

- Call the first string s_c .
- We first run edit distance between s_c and s_2 , which may introduce gaps in either string. Importantly, this maps every character in s_2 to a character in s_c .
- Then, we run edit distance between s_c and s_3 , which may introduce gaps in either string. This provides a mapping between s_3 and s_c . However, transitively, we also get a mapping from s_3 to s_2 .
- We keep repeating this process with each string. By the end, we have a mapping between every string, which can be used to rebuild an alignment matrix.
- Note that s_c does not have to be s_1 . In fact, we want to choose the best s_c , since a different “center” string would change the mappings. Most approximation algorithms using this heuristic focus on finding the best s_c .

Star MSA Approximation Algorithm

Definitions

- Let $d_M(s_i, s_j)$ represent the cost of the alignment between sequences s_i and s_j **implied by M** .
- Let $a(s_i, s_j)$ represent the cost of the optimal pairwise alignment between s_i and s_j . (This is just edit distance between the two strings.)

These are not the same. Consider the example $S = \{AT, A, T, AT, AT\}$.

Opt (a) between A and T:		d_M between A and T:	
A			A-
(mismatch)			(double gap insertion)
T			-T

We do the double gap insertion with d_M because the rest of the strings would pull the A to the left and T to the right.

Assumptions

1. $d_M(s_i, s_j)$ has the form

$$d_M(s_i, s_j) = \sum_{(u_i, v_j) \in M} \text{cost}(u_i, v_j)$$

2. $\text{cost}(u, v) < \text{cost}(u, w) + \text{cost}(w, v)$
3. $\text{SP-score}(M) := \sum_{i < j} d_M(s_i, s_j)$. In other words, the score of the entire matrix is the sum of all d_M pairings.

STAR algorithm

1. Build all $O(p^2)$ OPT pairwise alignments
2. Choose s_c to be the input string that minimizes

$$\sum_{i \neq c} a(s_c, s_i)$$

3. Do progressive d with s_c as anchor

We call this the STAR algorithm because by picking an s_c and aligning all the other strings to it, it looks like a star if each string is a node on the graph.

- An edge on the STAR graph between nodes representing s_i and s_c would be $a(s_c, s_i)$.

Note that with this formulation, **SP-score** is like the sum of every possible stars (divided by 2 since all edges are double-counted).

- **SP-score** is a clique containing all the nodes (or all strings)
- “like” the sum because **SP-score** edges represent $d_M(s_i, s_j)$, not $a(s_i, s_j)$.

Theorem:

- **STAR** represents the **SP-score** of M returned by the STAR algorithm
- **OPT** represents the **SP-score** of M^* optimal.
- $\frac{\text{STAR}}{\text{OPT}} \leq 2$

Proof: We want to prove that

- $\text{STAR} \leq 2B$
- $\text{OPT} \geq B$

We know by definition that

$$2 \cdot \text{STAR} = \sum_{i,j} d_{\text{STAR}}(S_i, S_j)$$

The sum part is the pairwise alignment implied by **STAR**. From here we can simplify:

$$\begin{aligned}
&\leq \sum_{i \neq j} [d_{\text{STAR}}(S_i, S_c) + d_{\text{STAR}}(S_c, S_j)] && \text{(by triangle inequality)} \\
&= \sum_{i \neq j} a(S_i, S_c) + a(S_c, S_j) && \text{(alg. did progressive alignment anchored at } S_c) \\
&= 2 \sum_{i \neq j} a(S_i, S_c) \\
&= 2 \sum_i \sum_j a(S_i, S_c) \\
&= 2 \sum_i a(S_i, S_c) \sum_j 1 \\
&\leq 2p \sum_i a(S_i, S_c) && (p \text{ is the number of sequences})
\end{aligned}$$

The above inequality is greater than $2 \cdot \text{STAR}$, so the 2 at the beginning cancel out.

$$\text{STAR} \leq p \sum_i a(S_i, S_c)$$

This is saying that the STAR algorithm can't give you worse than p copies of the alignment.

Now, define

$$B = \frac{1}{2}p \sum_i (S_i, S_c)$$

therefore,

$$\Rightarrow \text{STAR} \leq 2B$$

Now, we consider the optimal alignments.

$$\begin{aligned} 2 \cdot \text{OPT} &= \sum_{i,j} d_{\text{OPT}}(S_i, S_j) \\ &\geq \sum_{i,j} a(S_i, S_j) \end{aligned}$$

The above is true because the optimal pairwise alignment, OPT , cannot be better than the optimal overall alignment, a .

$$\begin{aligned} &= \sum_i \sum_j a(S_i, S_j) \\ &\geq \sum_i \sum_j a(S_i, S_c) && \text{The star centered at } c \text{ is the optimal star} \\ &= p \sum_i a(S_i, S_c) \end{aligned}$$

Now, recall that this above expression is equal to $2 \cdot \text{OPT}$, so we can again, divide both sides by 2.

$$\frac{1}{2}p \sum_i a(S_i, S_c)$$

This is equal to B , as defined before. Therefore, both $\text{STAR} \leq 2B$ and $\text{OPT} \geq B$, and $\frac{\text{STAR}}{\text{OPT}} \leq 2$.